

ME 375 Final Project – Robot Control

5/1/2025

Team Member	Report	Competition	Summary	Signature
Ryan Goffinet	80%	10%	Wrote most of the structure and formatting of the report, calibrated IR sensor and controller	
Gabriel Boyd	10%	60%	Wrote motor controllers, calibrated line following sensor, established structure for motor speed control	
Soumil Iyer	10%	30%	Wrote motor speed difference controller using root locus techniques	
Total	100%	100%	-	-

Introduction

The goal of this competition is to build an autonomous robot that moves around an unknown track layout for two laps as fast as possible and then park at 30 cm distance away from a wall. The target speed for the competition is for the robot to complete these two laps within 30 seconds. The track will consist of a double line that is 23.5/64" parallel to each other with a gap of 5/64" between.

To achieve this, a controller design for the system was created. The MIMO system, a robot, is built using parts from the AndyMark am-5150a skitter robot kit. This system includes two 12v gearmotors with encoders so that the position vs. time of each wheel is known, a line follower sensor to ensure the robot remains on the competition track, and an infrared sensor to determine the distance to the parking wall. The motor encoders have a 12 V input voltage with a 14:1 ratio. The line follower sensors have an optimal sensing distance of 3 mm, and supply 3 analog voltage measurements. Finally, the infrared sensor has a range of 10-80 cm. The robot was modeled as a dual-input dual-output system.

Subsystem Characterization and Full System Model

The robot system can be characterized by 4 major subsystems: the two encoder-gearbox-motor subsystems, the infrared sensor subsystem, and the line follower sensor subsystem. The first of these subsystems, the encoders, was first tested for the number of counts per revolution. This was achieved by manually turning each wheel 360 degrees 10 times, finding the total number of counts, and then taking the average of these measurements. This test resulted in 200 counts per rotation for motor 1 (the motor on the left side of the robot) and 195 counts per rotation for motor 2 (the motor on the right side of the robot). Assuming no slip, this results in 38 encoder counts per 1" travelled by the robot. This was then validated by Simulink oscilloscope.

The second major subsystem is the infrared sensor. The accuracy of the infrared sensor was first tested using a ruler. Using this data, there was found to be little error (less than 0.5 cm) between the sensor reading and the actual distance. These values were obtained using the default calibration constants of $a_1=-4.3019$, $k_1=239.1904$, $a_2=-0.6848$, and $k_2=48.1191$, using Equation 1 found below. This was tested by taking a continuous measurement from 10 to 80 cm, increasing and decreasing the distance up and down the ruler in order to account for potential hysteresis effects, none of which were observed. Using this data, a linear model for voltage and distance was obtained. This was then validated by selecting 5 points among the data obtained and testing if it fit the trendline. These results are shown in Figure 1.

$$D(v) = a_1 e^{k_1 v} + a_2 e^{k_2 v} \quad (\text{Eq. 1})$$

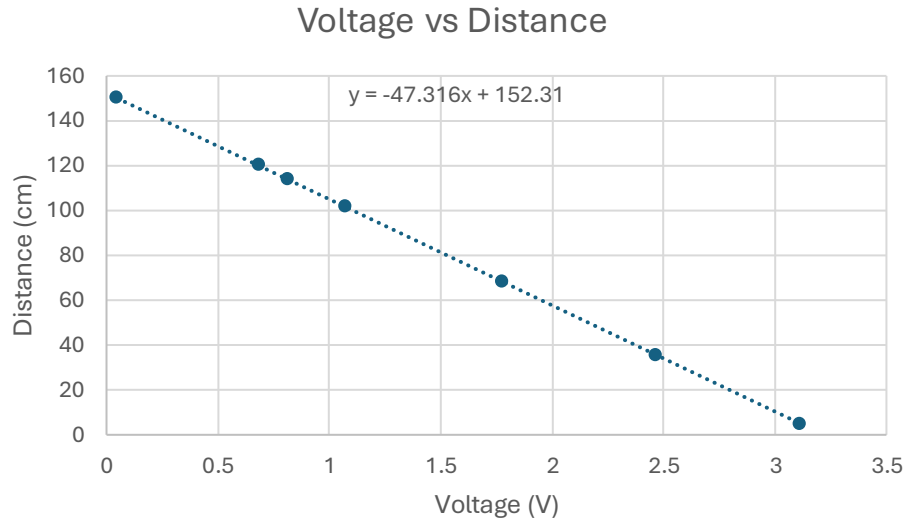


Figure 1 - Calibration Curve for IR Distance Sensor

The third major subsystem is the line follower sensor. The line follower sensor was first calibrated using the default binary threshold value of 300. The output voltages of each analog sensor were then compared to the horizontal y offset of the robot from the line. Using specified distances for each measurement, a linear regression model for three states were obtained: being on track, being too far to the left, and being too far to the right. The results of the calibration for the robot in the central regime are shown in Figure 2. The results of the calibration for the robot in the left regime are shown in Figure 3. The results of the calibration for the robot in the right regime of the robot are shown in Figure 4. These results were then validated using similar tests at random intervals and checking the fit to the linear regression.

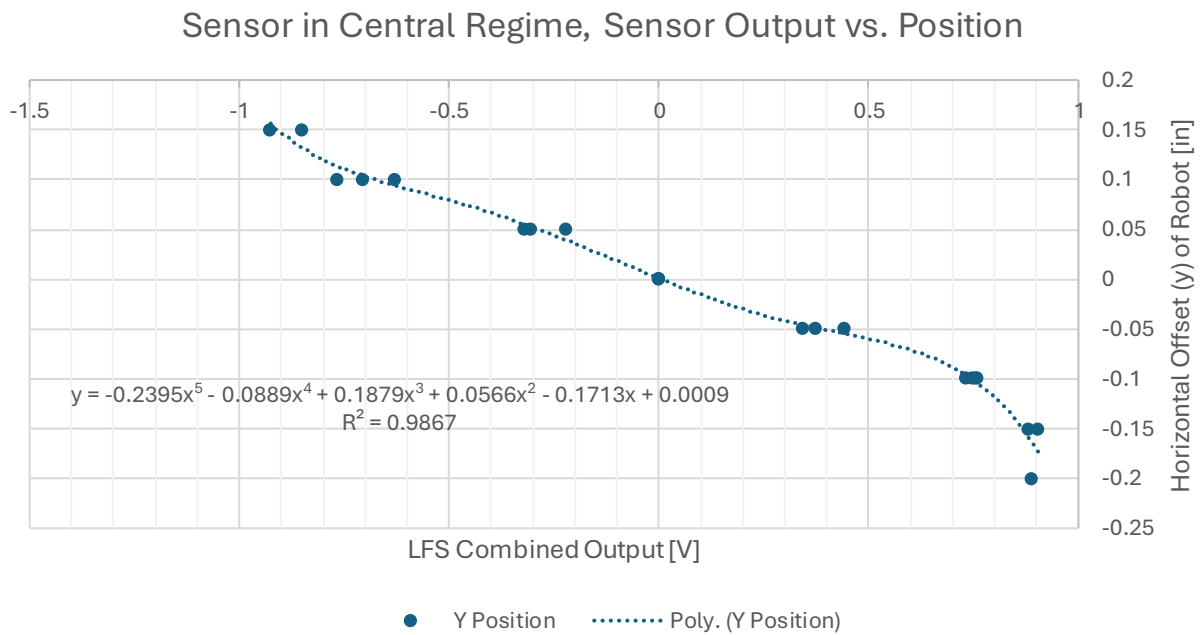


Figure 2 - Central Regime ($-0.2'' < \text{Line offset} < 0.2''$) Calibration Curve

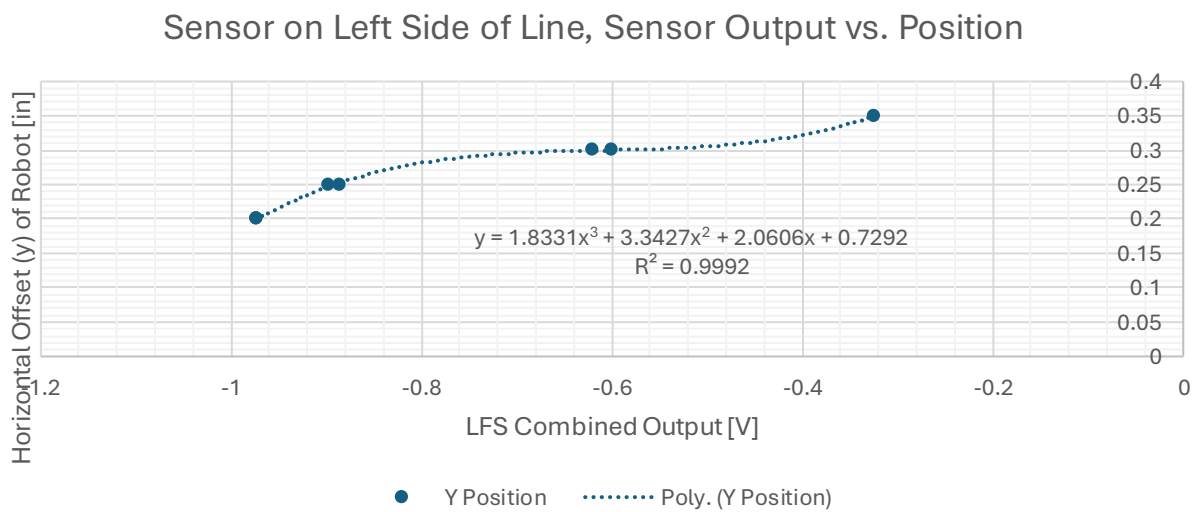


Figure 3 - Left Regime ($\text{Line offset} < -0.2''$) Calibration Curve

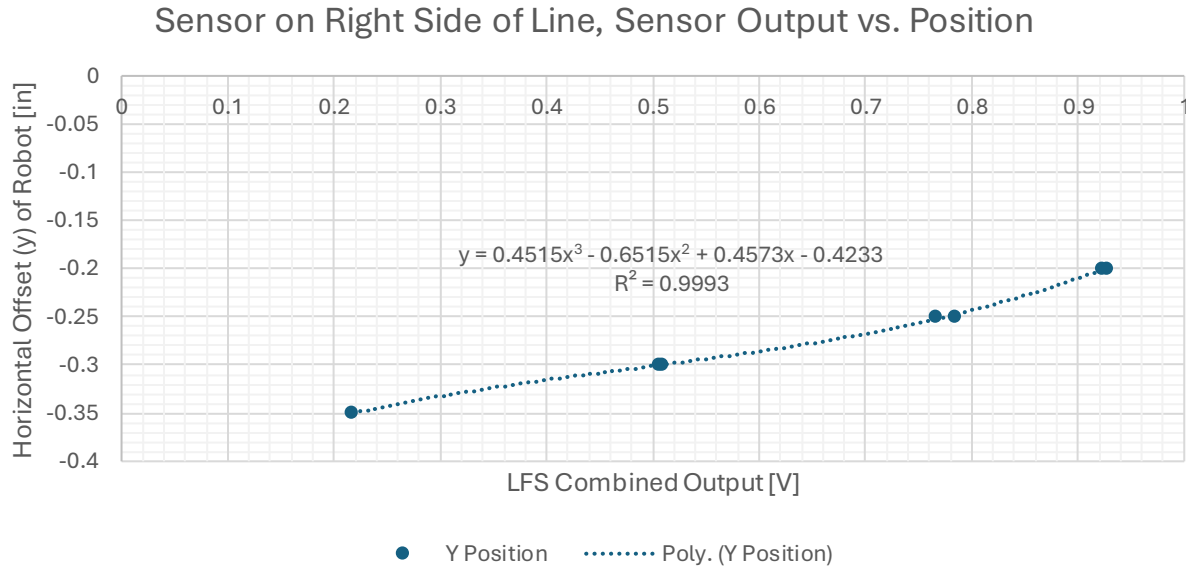


Figure 4 - Right Regime (line offset > 0.2") Calibration Curve

Finally, to create a controller for the robot's kinematics, a linear transfer function for the two motor-gearbox-wheel subsystems were obtained. These systems work like a first-order system, so the static gain K and time constant τ were obtained by using a pulse generator and finding the relationship between the forward speed and input voltage. The result of these tests can be found in Figure 5. These tests resulted in a static gain of 14.375 and 14.95, and a time constant of .1 and .08 for motors 1 and 2, respectively. A PWM cycle of 100% results in a speed of 5.13 in/s. These values were then validated using the scope function in the completed Simulink file. However, there are some nonlinearities for these values. In particular, in order to account for the static friction, the friction compensators have deadbands of .256296 and .304304 for motors 1 and 2, respectively.

Using the data from these four systems, a Simulink model was created to control the robot. The full system model is shown in Figure 5. The overall model takes the y position offset of the robot and calculates the difference in speed for each wheel needed to correct and follow the line. This information then feeds into the state machine to determine in what manner the system should use these speed values, depending on the state of the robot at the time. Then, this information is sent to the inner controller in order to correct the speeds of each wheel. This process then repeats for each sensor measurement obtained. When the state machine determines that the robot needs to park, the speed values are then calculated using a separate controller, that prioritizes minimizing overshoot when calculating the speed of the wheels in order to safely park within the desired range. These speed values are then once again sent through the state machine and to the inner controllers in the same manner as the other outer speed controller.

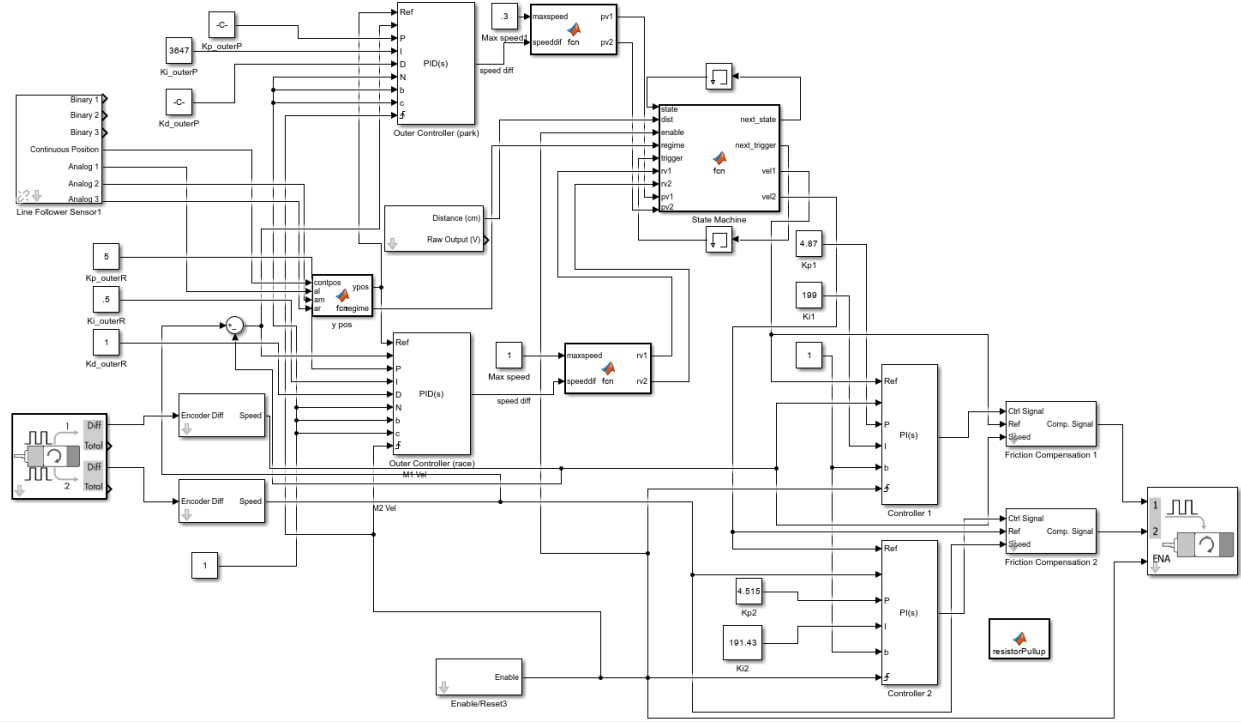


Figure 5 - Complete Simulink Model

The model's motor subsystems are characterized with Equation 2, where v_x is the velocity in the forward direction. The rotation away from the track is shown in Equation 3, where θ is shown as the angle away from the center line. Finally, y_{total} , the lateral offset from the track, is found using is found using the linear offset from the track and the distance from the center of mass to the wheel. This is shown in equation 4. Using these values, the corresponding speed of each wheel is obtained through the controller structure.

$$v_x = \frac{v_1 + v_2}{2} \quad (\text{Eq. 2})$$

$$\dot{\theta} = \frac{v_2 - v_1}{2h} \quad (\text{Eq. 3})$$

$$y_{total} = y + l \sin \theta \quad (\text{Eq. 4})$$

In the racing regime, the maximum wheel speed is manually set to the maximum attainable motor speed. The difference in wheel speeds was found using the Laplace transform of the time-domain system model. K could be empirically set depending on maximum speed to ensure race completion.

$$G(s) = \frac{\dot{y}}{x_2 - x_1} = \frac{K}{0.000784s^2 + 0.038s + 1} \quad (\text{Eq. 5})$$

The transfer functions for the motors themselves are modeled as biproper first-order poles of the form in Equation 6.

$$P(s)_{motor} = \frac{K}{\tau s + 1} \quad (\text{Eq. 6})$$

Gain and time constant values for each motor are as shown in Table 1.

Table 1 – Static Gain and Time Constant for Each Gearmotor Plant Model

Motor	LEFT	RIGHT
Static Gain K	14.375	14.95
Time Constant τ	0.08 seconds	0.1 seconds

Proposed Controller Structure and Design

The robot's planned control structure is a combination of PID and PI controllers in series. These controllers consist of the outer controller for calculating racing speeds, an outer controller for calculating parking velocity, and two inner controllers for changing the speeds of the wheels. The outer controllers are PID controllers, while the inner controllers are PI controllers. An overview of these controllers can be found in Figure 6. The team believes that a PID controller is the best fit for the outer controllers because of the significant number of nonlinearities that can occur from the sensor readings, especially the line follower sensor. On the other hand, the team believes that only PI controllers are necessary for the inner controllers because they are only correcting for the current speeds of the wheels.

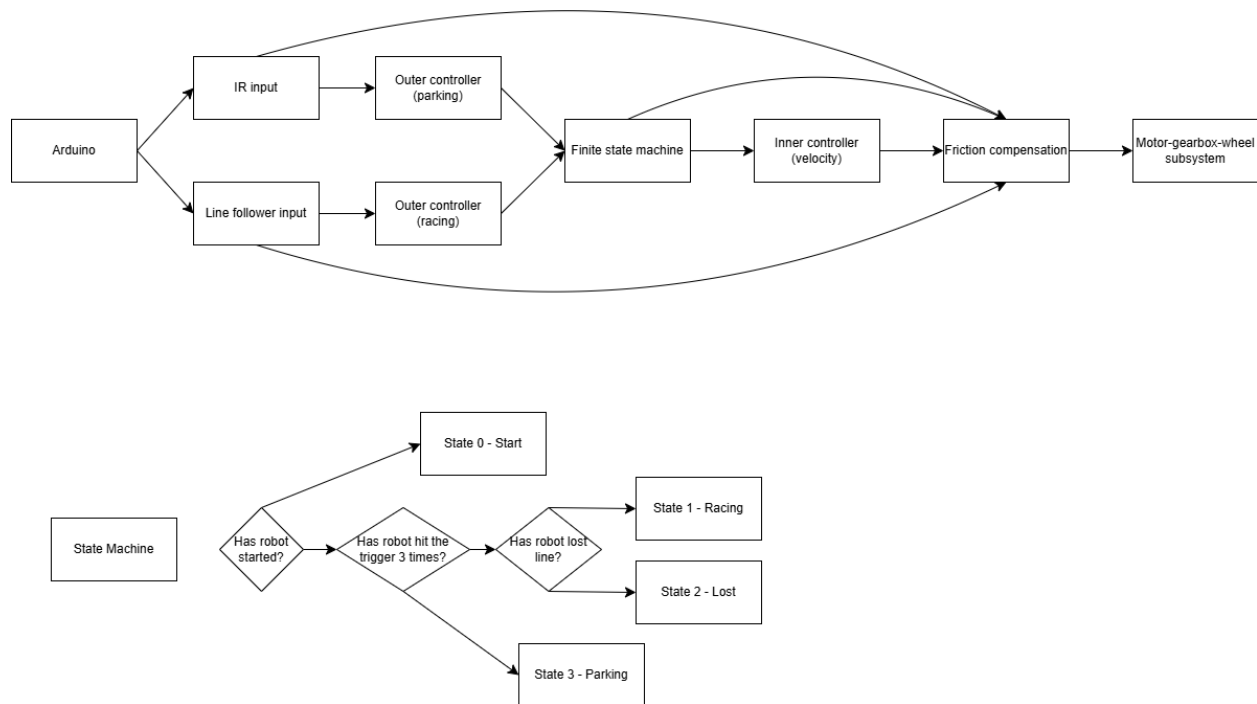


Figure 6 - Control Schematic

The overall logic for the control schematic is as follows: To begin, the Simulink code is uploaded to the Arduino stored on the robot. Then, the line follower sensor and IR sensors send voltage values that are converted to distance measurements using the linear regression models for each sensor. Then, this data is sent to their respective outer controllers, which calculate the difference in speeds each wheel need in order to stay on the line. These speed values are then sent to the finite state machine, which will determine which speed values from which controller to used based on the state the robot is currently in: racing velocity for the two laps, and parking

velocity for the 30 cm stop at the end. Finally, these speeds are sent to the inner controllers, which adjust the actual wheel speeds after accounting for the friction compensation.

The controller is based on a finite state machine with 3 main states: state 0 refers to the system being at rest, state 1 refers to the movement and navigation of the robot around the track, state 2 refers to when the robot does not have a clear direction to the path, and state 3 refers to the parking mechanism. The signal to transition from state 0 to state 1 is the activation of the Simulink model. The signal to transition from state 1 to state 3 occurs when the robot detects the black trigger mark at the start of the track three times, signifying that the robot has made 2 complete laps. The signal to transition from state 1 to state 2 is when the robot can no longer detect the line. The signal to transition back from state 2 to state 1 is when the robot recovers onto the track. Finally, the signal to transition from state 3 to state 0 is a successful stop of the robot from state 3. The trigger mechanism operates on the principle that the trigger mark is the only place on the track that all 3 analog signals from the line follower sensor will return a value of 1. Therefore, whenever this occurs, the trigger count is incremented, and a 2 second timer is started to ensure that the trigger is not counted multiple times. In order to prevent the robot from straying too far from the track, state 3 enables a separate form of navigation that stops the robot and allows it to pivot until it finds the track and returns to it again.

The outer controller for racing velocity uses the analog signals from the line follower sensor and uses a linear regression model in order to find the y offset from the center of the line. This resulting y position is then used to calculate the necessary speed for each wheel based on the max speed. After the finite state machine chooses these values, the racing velocities are used as the reference for the inner controllers. The inner controllers determine how to best change the speed to fit the reference. The outer controller for racing velocity was designed using the “rltool” on MATLAB which is mentioned in Equation 5. It uses the difference of the two motor velocities as the signal and the y position value as its reference. This controller prioritizes a quick rise time and low percent overshoot in order to maximize the speed of the robot.

The outer controller for parking velocity uses the analog signal from the infrared sensor to determine how far the robot still needs to move in order to park in the correct area. It uses a linear regression model from the calibration tests in order to find the distance remaining to the stopping point, and returns the proper velocity based on that distance. After the finite state machine chooses these values, the racing velocities are used as the reference for the inner controllers. The outer controller for parking velocity uses a PID controller with $K_p = 1203.51$, $K_i = 3647$, and $K_d = 397.16$. These values were obtained using the rltool function in MATLAB by using the plant function from the IR sensor calibration, shown in Equation 7. This controller’s main goal was to minimize the percent overshoot to less than 2 percent in order to ensure that the robot stops precisely. It uses the difference of the two motor velocities as the signal and the y position value as the reference.

$$P = \frac{1}{152.31s - 47.316} \quad (\text{Eq. 7})$$

The feedback loop implemented is for the motor kinematics. The objective for these controllers is to determine the speed for each wheel. These controllers were chosen to be PI controllers because they are only concerned with changing the speed of each wheel from each calculated value from the outer PID controller. The K_p values are 4.87 and 4.515, and the K_i values are 199 and 191.43 for the first and second motors, respectively. They use the velocity

values from the encoders as the signal and use the desired velocity values from the finite state machine as the reference. Finally, the friction compensators use the result of the PI controllers as the signal, the velocity from the finite state machine as the reference, and the speed from the motor encoders as the current speed.

Potential Problems

One potential problem that may arise from the desired control scheme is chattering, as this could result in a significant slowdown on the straight sections of the track. This can be resolved through the addition of anti-windup mechanisms like an integrator or through controller tuning. Empirically, the integral term could also be decreased, which decreases the dampened frequency of the overall position-motor speed transfer function. Another potential problem that may arise is error due to delay. To account for this, a Smith predictor could be designed and implemented to minimize that error. Finally, there are bound to be errors within the integration of multiple controllers into one system. To circumvent this, each part of the system can be tested and validated individually before adding each new part into the system, ensuring that no major errors occur when the full system is implemented.